

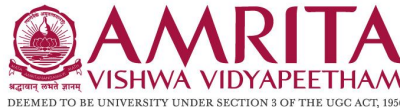
Real-Time Network Intrusion Detection System Using Big Data and Deep Learning

Final Project Presentation

Group C-17

Amrita School of Artificial Intelligence, Amrita Vishwa Vidyapeetham, Coimbatore

March 2026



Motivation & Problem Statement

Why This Project?

- Enterprise networks generate traffic at big data scale — millions of flows/days
- Signature-based IDS cannot detect novel or zero-day attacks
- Prior ML-based IDS work trains and tests in static batch mode only — no real-time deployment, no end-to-end validation

This Project Addresses All Three Gaps

1. **Scale:** Distributed Spark/HDFS pipeline on 16.2M real network flows
2. **Accuracy:** Compare LR, CNN, LSTM with explicit imbalance handling
3. **Deployment:** Live Kafka → Spark Streaming → MongoDB pipeline with validated accuracy

Objectives

1. **Distributed Preprocessing:** Spark/HDFS for 16.2M records; 80 → 29 features via RF; Benign downsampling; minority oversampling
2. **Multi-Model Evaluation:** Spark MLlib Logistic Regression · Keras Convolutional Neural Network (CNN) · Keras Long Short-Term Memory (LSTM)
3. **Real-Time Deployment:** Kafka → Spark Streaming → MongoDB; validate streaming vs batch accuracy; attack alerts with priority classification

Related Work & Research Gap

Work	Contribution	Limitation
Buczak & Guven (2016)	ML survey for IDS; class imbalance as key challenge	Batch only
Sharafaldin et al. (2018)	CIC-IDS-2018: realistic 14-attack benchmark	No ML pipeline
Basnet et al. (2019)	CNN/LSTM outperform LR on minority classes	No deployment
Hagar & Gawali (2022)	Spark + deep learning for scalable IDS	No streaming pipeline
This work	Distributed training + live streaming + accuracy validation	—

No prior work combines distributed Spark training, real-time Kafka streaming deployment, and end-to-end batch-vs-streaming accuracy validation on CIC-IDS-2018.

CSE-CIC-IDS2018 Dataset & Class Imbalance

Dataset Facts

- **16,232,943** raw records
- **10** capture days
- **80** CICFlowMeter features
- **15** classes: Benign + 14 attacks (DDoS ×3, DoS ×4, BruteForce ×4, Bot, Infiltration, Web)
- Benign = **83%** of raw data
SQL Injection = **87** records
⇒ 155,000:1 ratio

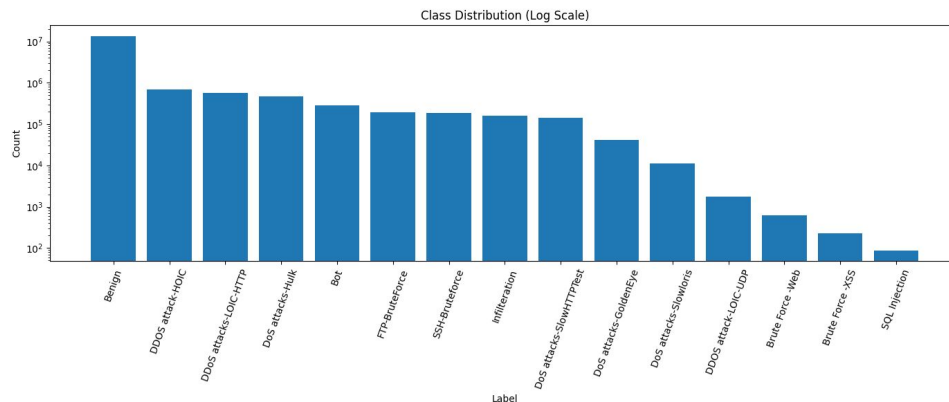


Figure: Raw class distribution (log scale)

HDFS: Distributed Storage Layer

Role of HDFS

- Raw CSVs total ~6.9GB across 10 files — too large for single-machine pandas workflows
- HDFS distributes storage and exposes a unified filesystem to Spark via `hdfs://namenode:8020` — no data movement needed at training or inference time
- NameNode handles metadata + directory tree (Web UI: `localhost:9870`); DataNode handles actual block storage (Web UI: `localhost:9864`)
- Block size: 128MB | Replication factor: 1 (single DataNode setup)
- All data bind-mounted to `./data/hdfs-namenode` and `./data/hdfs-datanode` — persists across docker compose down

The figure shows two screenshots of the HDFS Web UI. The top screenshot displays the directory listing for `/user/spark/ids`, showing three files: `models`, `processed`, and `raw`. The bottom screenshot displays the directory listing for `/user/spark/ids/processed`, showing eleven files: `class_distribution`, `label_mapping`, `parquet_balanced`, `parquet_by_file`, `parquet_clean`, `parquet_clean_clipped`, `parquet_encoded`, `parquet_merged`, `parquet_time`, `rf_feature_importance`, and `splits_stratified`. Both screenshots include a search bar and pagination controls.

Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name
drwxr-xr-x	root	supergroup	0 B	Feb 21 23:19	0	0 B	models
drwxr-xr-x	root	supergroup	0 B	Feb 20 23:28	0	0 B	processed
drwxr-xr-x	root	supergroup	0 B	Feb 19 12:03	0	0 B	raw

Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name
drwxr-xr-x	root	supergroup	0 B	Feb 20 22:39	0	0 B	class_distribution
drwxr-xr-x	root	supergroup	0 B	Feb 20 22:51	0	0 B	label_mapping
drwxr-xr-x	root	supergroup	0 B	Feb 20 22:47	0	0 B	parquet_balanced
drwxr-xr-x	root	supergroup	0 B	Feb 19 15:36	0	0 B	parquet_by_file
drwxr-xr-x	root	supergroup	0 B	Feb 19 15:46	0	0 B	parquet_clean
drwxr-xr-x	root	supergroup	0 B	Feb 20 22:38	0	0 B	parquet_clean_clipped
drwxr-xr-x	root	supergroup	0 B	Feb 20 22:51	0	0 B	parquet_encoded
drwxr-xr-x	root	supergroup	0 B	Feb 19 15:40	0	0 B	parquet_merged
drwxr-xr-x	root	supergroup	0 B	Feb 20 22:54	0	0 B	parquet_time
drwxr-xr-x	root	supergroup	0 B	Feb 20 23:28	0	0 B	rf_feature_importance
drwxr-xr-x	root	supergroup	0 B	Feb 20 22:57	0	0 B	splits_stratified

Figure: Usage of HDFS (snapshot: Directory)

Apache Spark & Kafka

Why Spark?

- **In-memory computation:** 10–100× faster than MapReduce for iterative ML
- **Unified API:** DataFrames + MLlib + Structured Streaming
- **Pipeline API:** VectorAssembler → StandardScaler → LR saved as single PipelineModel object

Kafka (KRaft Mode)

- **KRaft mode:** no ZooKeeper — built-in Raft consensus
- **Topic:** network-traffic
- 1 partition (single broker), replication factor 1
- **Log retention:** 24 hours

Spark Configuration

- Master daemon memory: 512MB
- Worker: 1536MB advertised, 2 cores, 2GB Docker hard cap
- Executor: 1GB heap, 1 core
- Shuffle partitions: 8 (tuned for 5.75M records)
- Network timeout: 300s | Heartbeat interval: 60s

Why Kafka?

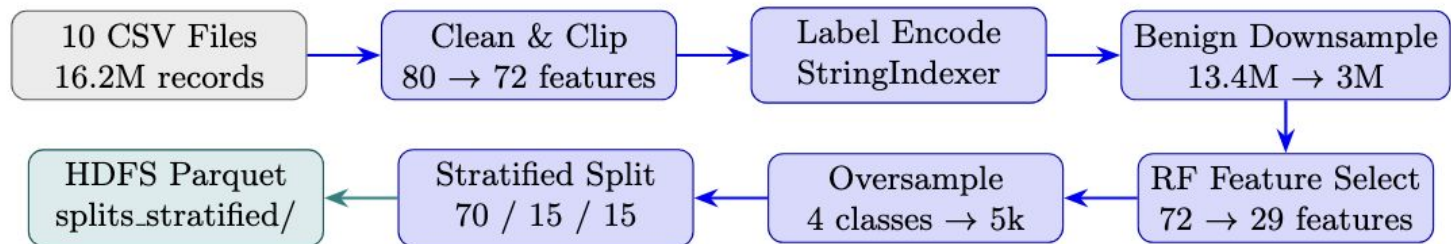
- High throughput
- Fault-tolerant replicated commit log
- Producer/consumer decoupling — producer and consumer run independently

MongoDB

Why MongoDB?

- **Schema-free documents** — each prediction record has varying fields depending on batch context; no rigid table schema needed
- **Write-heavy workload** — `insert_many()` per micro-batch (every 5 seconds); MongoDB handles high-frequency writes better than a relational DB at this scale
- **No joins needed** — predictions and alerts are self-contained documents; no relational structure required
- **Easy to query** — aggregations like attack counts by class, high-priority filtering, and batch-level accuracy all done in mongosh with simple queries
- **Docker-friendly** — single container, zero config, bind-mounted to `./data/mongodb`

Preprocessing Pipeline



Steps

1. Drop 59 duplicate header rows; remove 8 all-zero features
2. Clip outliers at 1st/99th percentile per feature
3. StringIndexer: label → integer (0-14)
4. Benign downsampled 13.4M→3M; all 14 attacks kept intact
5. RF importance top 30; **exclude** timestamp_unix (rank 4) to prevent temporal leakage → **29 features**
6. Oversample 4 tiny classes to 5,000 each; stratified 70/15/15 → Parquet on HDFS

Final Split Sizes

Splits	Records	%
Train	4,026,287	70
Val	861,484	15
Test	863,597	15
Total	5,751,368	100

Feature Engineering & Selection

Random Forest Selection

- Train RF on balanced sample
- Rank 72 features by importance
- Select top 30 features
- **Exclude** timestamp_unix (rank 4)
- Final: **29** features

Rank	Feature	Description
1	Init Fwd Win Byts	Initial window size (forward)
2	Dst Port	Destination port number
3	Fwd Seg Size Min	Minimum forward segment size
4	Fwd Pkt Len Max	Maximum forward packet length
5	Fwd Header Len	Total forward header length
6	Subflow Fwd Byts	Mean forward bytes per subflow
7	TotLen Fwd Pkts	Total forward packet payload
8	Flow Duration	Duration of the flow
9	Flow IAT Max	Maximum inter-arrival time
10	Fwd IAT Min	Minimum forward IAT

Why Exclude timestamp_unix?

Prevents temporal data leakage — models would identify attack-heavy days by time rather than network behavior

After Balancing: Train/Val/Test Distribution

Imbalance Strategy

- Oversampling + inverse-frequency weighting: $w_c = \min(N / K \cdot n_c, 15)$, Benign floored at 0.4

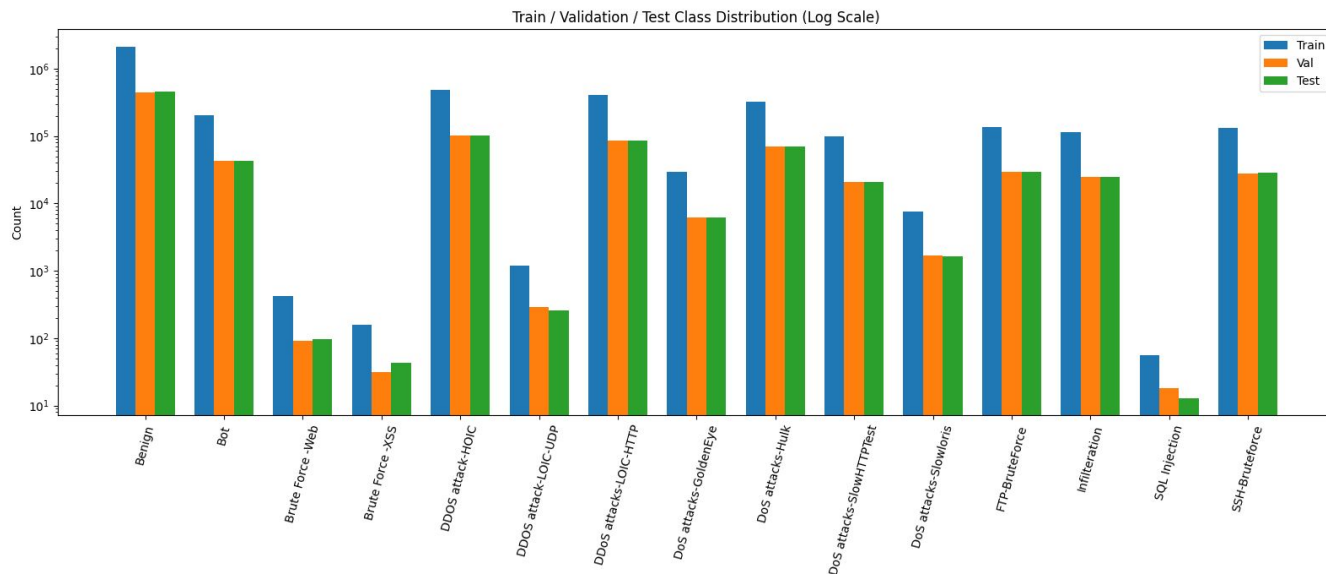
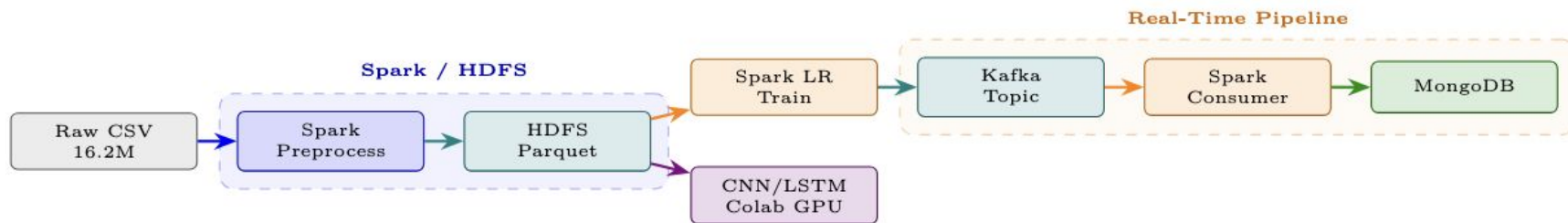


Figure: Per-class record counts across splits after Benign downsampling and minority oversampling (log scale). Stratification ensures consistent class ratios.

End-to-End Architecture



Docker Compose Stack

- HDFS NameNode + DataNode (768MB each, replication=1)
- Spark Master (512MB daemon) + Worker (1536MB advertised, 2GB Docker cap)
- Spark-Jupyter (1536MB, driver 1GB)
- Kafka KRaft single broker (768MB cap, JVM heap 512MB max)
- MongoDB 6.0 (384MB)
- All storage bind-mounted to `./data/` — data persists across restarts

Why LR for Streaming?

CNN/LSTM require Keras + sklearn inside `foreachBatch` — significant overhead. Spark LR is a native `PipelineModel`: one `model.transform()` call, zero extra dependencies, loads directly from HDFS.

Model 1: Spark Logistic Regression

MLlib LR Pipeline

- VectorAssembler (29 features)
StandardScaler
LogisticRegression (multinomial)
- regParam=0.01, elasticNet=0.5
- Converged in 91 iterations (max 150)
- weightCol = inverse-frequency class weights

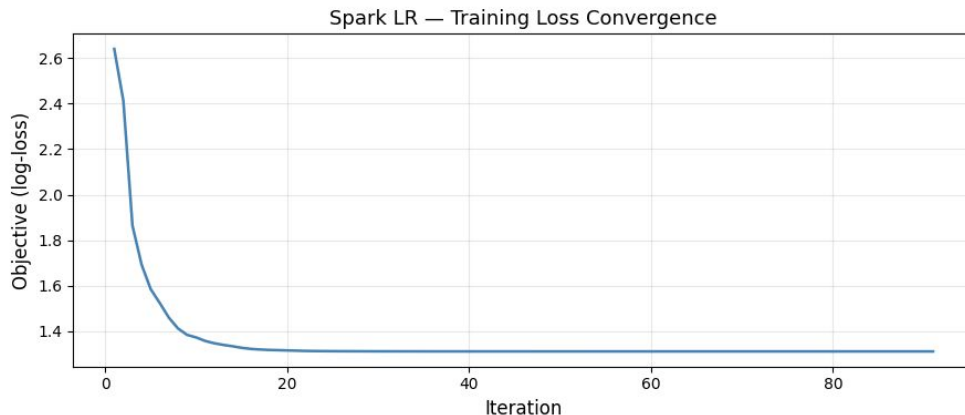
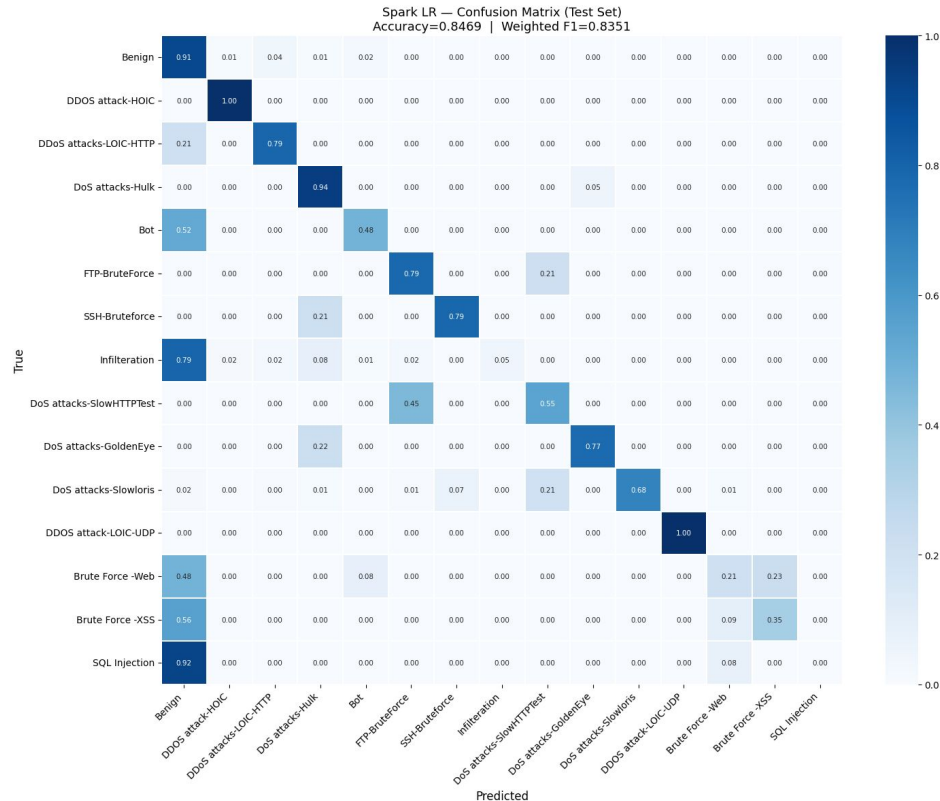


Figure: Training loss over 91 iterations (15.32 min)

Advantage

Native Spark deployment — `model.transform()` in streaming consumer, no external dependencies

Model 1: Spark Logistic Regression



Test Results

Accuracy **84.69%**

Weighted F1 0.8351

Weighted Precision 0.8416

Weighted Recall 0.8469

Figure: LR confusion matrix – strong on DDoS/DoS; Infiltration misclassified as Benign

Model 2: Keras Convolutional Neural Network (CNN)

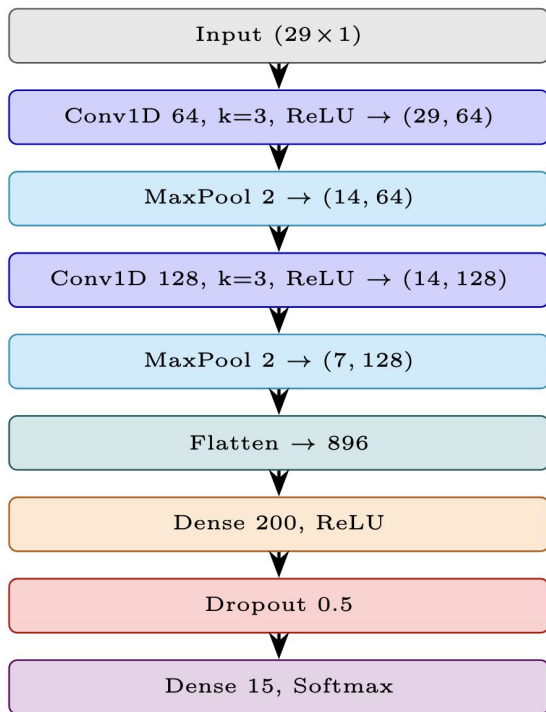


Figure: CNN architecture

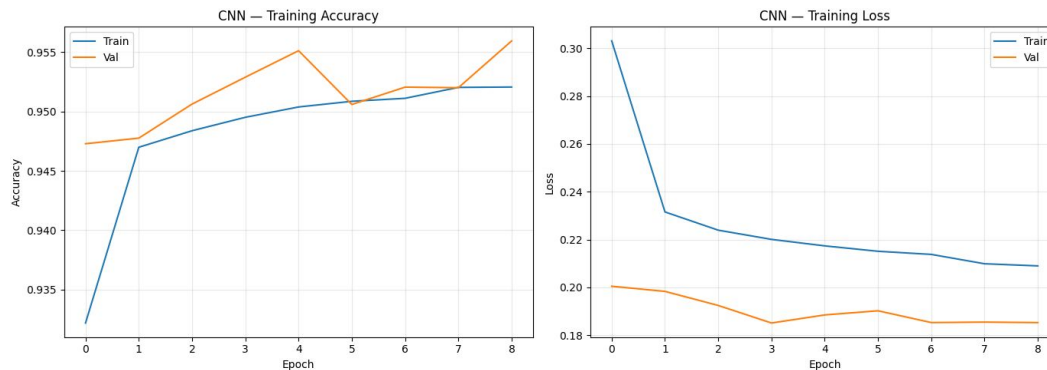
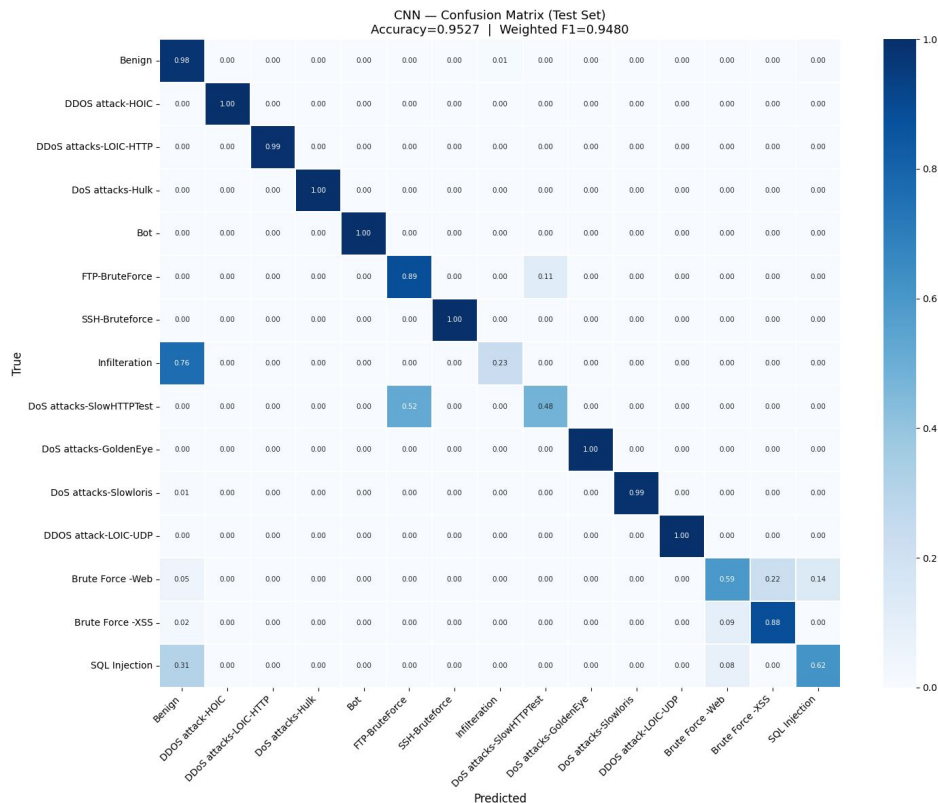


Figure (above): CNN training vs. validation curves

Training Configuration

- Adam lr=0.0001
- Categorical crossentropy
- Batch 256
- EarlyStopping patience=5
- 9 epochs (best: ep.4)
- 20.22 min T4 GPU

Model 2: Keras Convolutional Neural Network (CNN)



Test Results	
Accuracy	95.27%
Weighted F1	0.9480
Weighted Precision	0.9500
Weighted Recall	0.9527

Figure: CNN confusion matrix – near-perfect on all common classes; only XSS and SQL remain weak

Model 3: Keras Long-Short Term Memory (LSTM)

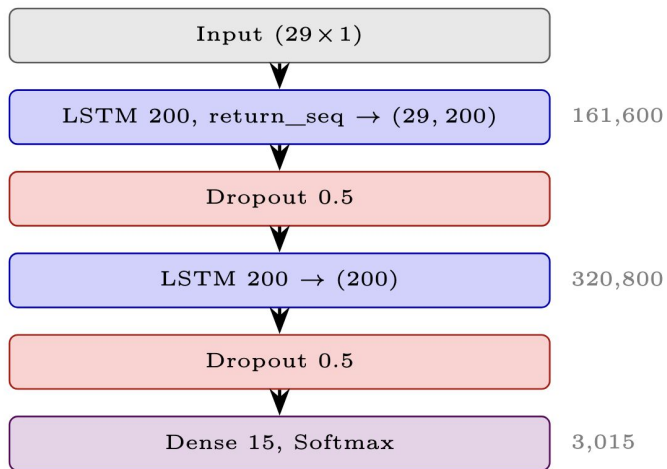


Figure: LSTM architecture

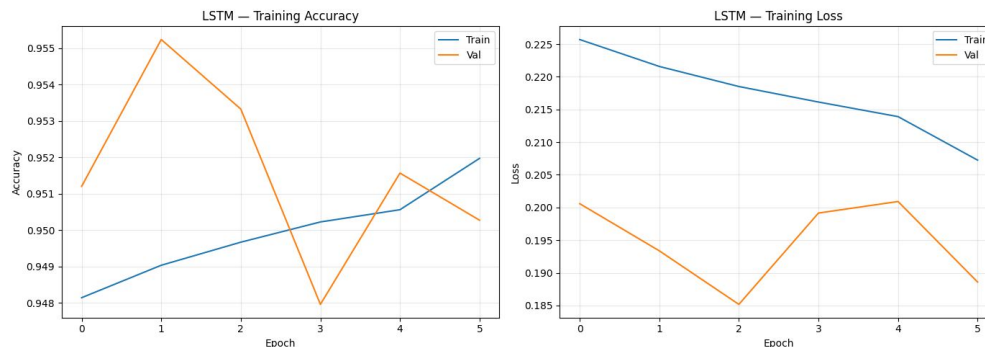
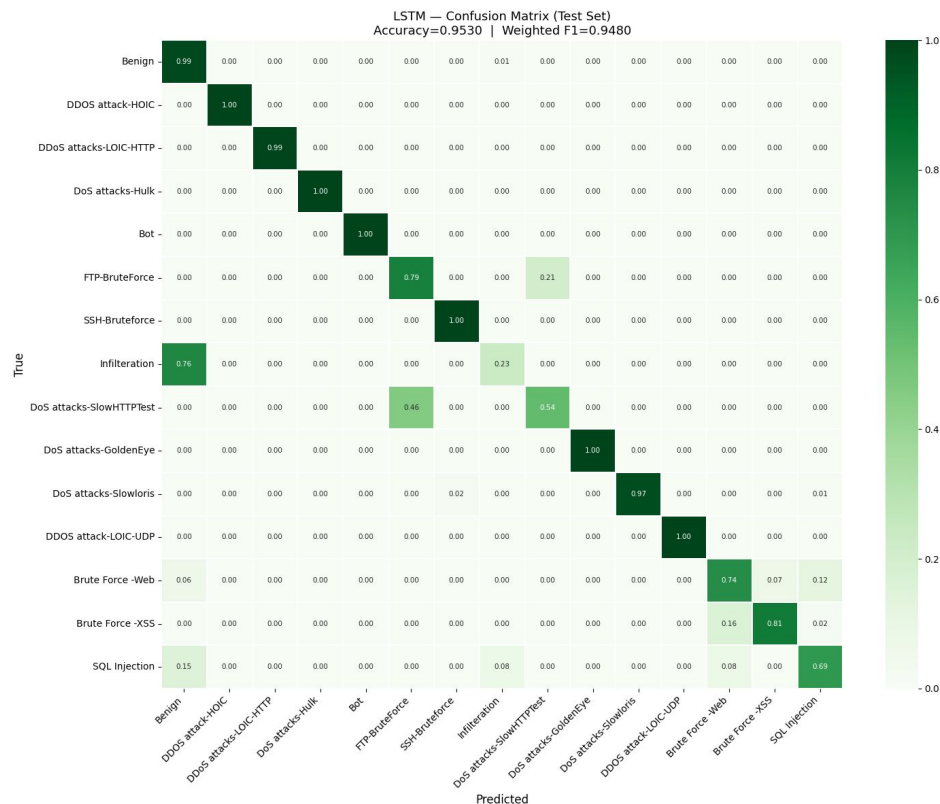


Figure (above): LSTM training vs. validation curves

Training Configuration

- Same as CNN
- 50 max epochs (best: ep.12)
- T4 GPU

Model 3: Keras Long-Short Term Memory (LSTM)



Test Results

Accuracy **95.3%**

Weighted F1 **0.9480**

Weighted Precision **0.9484**

Weighted Recall **0.9530**

Figure: LSTM confusion matrix – near-perfect diagonal on 10 of 15 classes; same confusion as CNN

Model Comparison & Per-Class Analysis

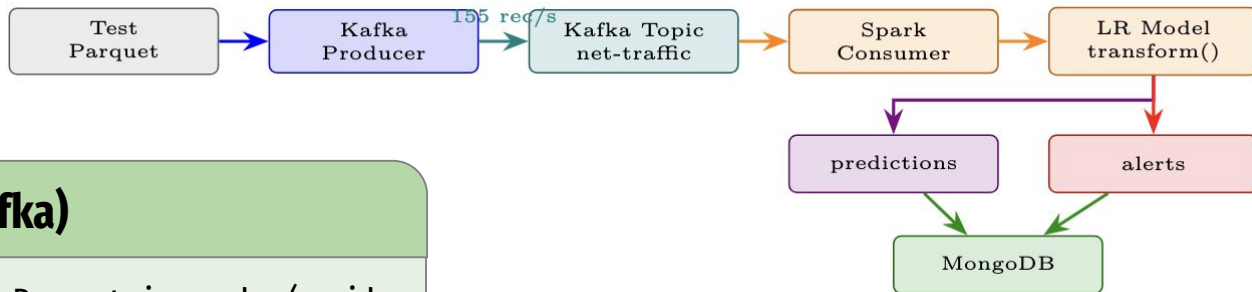
Key Findings

- CNN and LSTM significantly outperform LR on most attack classes.
- Especially strong performance (>0.99 F1 scores) for DDoS/DoS variants, Bot, and GoldenEye attacks.
- Infiltration, Brute Force -XSS, and SQL Injection remain challenging across all models.
- Difficulty is attributed to limited test samples and network signatures overlapping with benign traffic.

Model	Accuracy	Weighted F1	Macro F1
Spark LR	84.69%	0.8351	0.5582
CNN	95.27%	0.9480	0.7253
LSTM	95.30%	0.9480	0.7229

Attack Class	LR F1	CNN F1	LSTM F1
Benign	0.8907	0.9715	0.9735
DDOS attack-HOIC	0.9749	0.9999	0.9999
DDoS attacks-LOIC-HTTP	0.7855	0.9957	0.9960
DoS attacks-Hulk	0.8843	0.9996	0.9994
Bot	0.5625	0.9973	0.9985
SSH-Bruteforce	0.8768	0.9990	0.9986
Infiltration	0.0856	0.3340	0.3368
DoS attacks-GoldenEye	0.6465	0.9977	0.9959
Brute Force -XSS	0.0215	0.0844	0.0857
SQL Injection	0.0000	0.0082	0.0251

Real-Time Streaming Pipeline



Producer (Kafka)

- Reads test Parquet via pandas (avoids competing with Spark worker for cores)
- Serializes each record as JSON, publishes to network-traffic topic
- acks=1, retries=3, linger=10ms, 5ms inter-record delay
- Achieved **~155 records/second**, 0 send failures
- **Total sent:** 708,779 records

Consumer (Spark Streaming) + MongoDB

- Connects to kafka:9092 (internal Docker network)
- startingOffsets=latest, maxOffsetsPerTrigger=10,000
- 5-second micro-batch trigger
- Loads LR PipelineModel from HDFS, applies via model.transform()
- **Adds per-record:** predicted_label, true_label, correct, is_attack, is_high_priority (7 DDoS/DoS classes), detection_time, batch_id
- Writes to predictions + alerts collections in **MongoDB** via pymongo

Streaming Pipeline Validation

Metric	Value
Total predictions	708,779
Correct predictions	590,700
Streaming accuracy	83.34%
Batch test accuracy	84.69%
Accuracy gap	1.35%
Total alerts generated	380,839
Producer throughput	~155 rec/s
Consumer trigger interval	5 seconds

Streaming accuracy within 1.35% of batch result; pipeline validated end-to-end. Gap explained by class-partitioned parquet ordering in the producer (test split sorted by class, so early micro-batches see non-representative distribution) — not a pipeline error.

```
base ~/Library/CloudStorage/OneDrive-Personal/College Stuff/Sem 6/Projects/CS_BDA/ids-project
docker exec -it mongodb mongosh ids_results
Current Mongosh Log ID: 69a9b495e96361ddda4f87fd
Connecting to: mongodb://127.0.0.1:27017/ids_results?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.5.8
Using MongoDB: 6.0.26
Using Mongosh: 2.5.8

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
The server generated these startup warnings when booting
2026-03-05T16:45:39.525+00:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
2026-03-05T16:45:39.525+00:00: Soft rlimits for open file descriptors too low
-----

ids_results> var total = db.predictions.countDocuments()
ids_results> var correct = db.predictions.countDocuments({correct: true})
ids_results> print ("Total verifiable: ", total)
Total verifiable: 708779
ids_results> print("Correct: ", correct)
Correct: 590700
ids_results> print("Streaming accuracy: ", (correct/total*100).toFixed(2) + "%")
Streaming accuracy: 83.34%

ids_results> db.alerts.countDocuments({ is_high_priority: true })
295732
ids_results> var atk_correct = db.alerts.countDocuments({ correct: true })
ids_results> var atk_total = db.alerts.countDocuments()
ids_results> print("Attack detection accuracy:", (atk_correct/atk_total*100).toFixed(2) + "%")
Attack detection accuracy: 84.78%
```

Figure: mongosh session confirming 708,779 predictions, 590,700 correct, streaming accuracy 83.34%, alert detection accuracy 84.78%

Key Contributions & Future Work

What We Built & Achieved

- **Preprocessing (Spark/HDFS):** 16.2M → 5.75M balanced records; 80 → 29 features (RF); Benign downsampling + minority oversampling + inverse-frequency weighting.
- **Models:** LR 84.69%; CNN 95.27%; LSTM 95.30%. CNN recommended: equivalent accuracy to LSTM, 6× faster.
- **Streaming:** 83.34% accuracy on 708,779 live records; 380,839 alerts; 1.35% gap from batch. End-to-end pipeline validated.

Future Directions

- Packet-level features to improve Infiltration, Brute Force-XSS, SQL Injection detection (flow-level features insufficient for these classes)
- TensorFlow Serving sidecar to deploy CNN/LSTM in the streaming pipeline without foreachBatch overhead
- Online learning for incremental model updates as new attack patterns emerge
- Grafana dashboard for real-time alert visualisation directly from MongoDB

References

1. Buczak & Guven (2016) — A Survey of Data Mining and Machine Learning Methods for Cyber Security Intrusion Detection — IEEE Communications Surveys & Tutorials 18(2)
2. Sharafaldin, Lashkari, Ghorbani (2018) — Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization — ICISSP 2018
3. Basnet et al. (2019) — Towards Detecting and Classifying Network Intrusion Traffic Using Deep Learning Frameworks — Journal of Internet Services and Information Security 9(4)
4. Hagar & Gawali (2022) — Apache Spark and Deep Learning Models for High-Performance Network Intrusion Detection Using CSE-CIC-IDS2018 — Computational Intelligence and Neuroscience
5. Zaharia et al. (2016) — Apache Spark: A Unified Engine for Big Data Processing — Communications of the ACM 59(11)
6. Kreps, Narkhede, Rao (2011) — Kafka: A Distributed Messaging System for Log Processing — NetDB Workshop